

Test-Driven Development, Mocking, and Dependency Isolation

Week 4 Lecture — SE 413/513: Software Testing

Dr. Kamrul Hasan

Grand Valley State University

Fall 2026

Learning Objectives

By the end of today's class, you should be able to:

- Explain the **Red-Green-Refactor** cycle and apply it to build a function incrementally.
- Explain why writing tests *first* tends to produce more testable designs.
- Identify hard-coded dependencies and refactor them using **dependency injection**.
- Define and distinguish the five kinds of **test double**: dummy, stub, fake, spy, mock.
- Explain the difference between **state verification** and **behavior verification**.
- Contrast the “classical” (Detroit) and “mockist” (London) schools of TDD.
- Recognize the risks of over-mocking and mocks that don't match real dependency behavior.

Part 1

Test-Driven Development

What Is Test-Driven Development?

Test-driven development (TDD): write a *failing* test *before* you write the production code that makes it pass.

- Traditional order: write code → (maybe) test it afterward.
- TDD inverts this: the test comes first, and **drives the design** of the code that follows.

Background

Kent Beck popularized TDD as part of **Extreme Programming** (1999), then wrote *Test-Driven Development: By Example* (2002). He describes it less as an invention than a rediscovery of an older, informal discipline — but his books are what turned it into a widely taught, named practice.

The Red-Green-Refactor Cycle

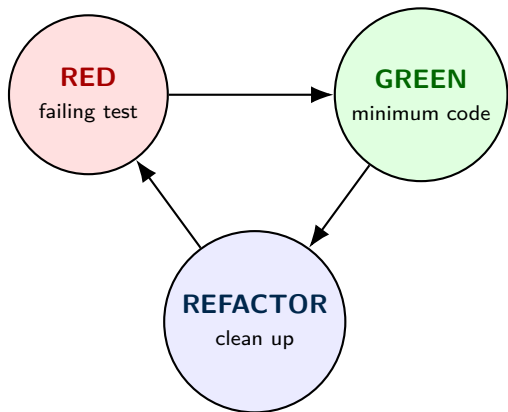
TDD is organized around a short, repeating cycle:

- 1 **Red:** Write a small test for behavior that doesn't exist yet. Watch it fail.
- 2 **Green:** Write the *minimum* code to make it pass — no more.
- 3 **Refactor:** Clean up under the safety net of a passing suite. Re-run tests after every change.

Tip

A test you have never watched *fail* is a test you cannot fully trust to catch a real defect. Always confirm the Red step for the right reason.

The Red–Green–Refactor Cycle



- 1 **Red:** Write a small test for behavior that doesn't exist yet. Watch it fail. A test you've never seen fail is one you can't fully trust.
- 2 **Green:** Write the *minimum* code to pass — no more.
- 3 **Refactor:** With tests green, clean up the code (remove duplication, improve names). Re-run tests after every change.

Key Idea: “Fake It Till You Make It”

Key Idea

Write only the *minimum* code to pass the current test — resist building out “obviously needed” functionality all at once. Kent Beck calls a deliberately oversimplified first implementation (even a hard-coded constant) “**fake it till you make it.**” The *next* test forces you to generalize.

This discipline is what keeps each cycle short, and each test meaningfully tied to one piece of behavior.

Worked Example: A Password Validator

Goal: `is_strong_password(password)` returns `True` only if the password:

- is at least 8 characters long,
- contains at least one digit, and
- contains at least one uppercase letter.

We will *not* write this all at once — we build it up one small Red-Green cycle at a time.

Cycle 1: Reject a Short Password

Red — a test we know should fail today:

```
def test_rejects_short_password():  
    assert is_strong_password("Ab1") is False
```

Green — the simplest thing that passes (“fake it”):

```
def is_strong_password(password):  
    return False
```

Not a complete implementation — but honest. Nothing yet requires more.

Cycle 2: Accept a Good Password

Red — this breaks the fake implementation:

```
def test_accepts_long_password_with_digit_and_uppercase():  
    assert is_strong_password("Abcdefg1") is True
```

Green — forced to add real logic:

```
def is_strong_password(password):  
    if len(password) < 8:  
        return False  
    return True
```

Cycles 3–4: The Same Pattern Continues

Cycle 3 adds a test/check for the digit rule; Cycle 4 adds one for the uppercase rule. Same Red → Green pattern each time:

```
def is_strong_password(password):  
    if len(password) < 8:  
        return False  
    if not any(char.isdigit() for char in password):  
        return False  
    if not any(char.isupper() for char in password):  
        return False  
    return True
```

Full test code for Cycles 3–4 is in the reading (Section 3.2) — omitted here for time.

Refactor: Clean Up With a Green Safety Net

All four tests pass. *Now*, and only now, clean up:

```
def is_strong_password(password):  
    rules_satisfied = [  
        len(password) >= 8,  
        any(char.isdigit() for char in password),  
        any(char.isupper() for char in password),  
    ]  
    return all(rules_satisfied)
```

Re-run all four tests to confirm nothing broke — they remain a small regression suite, a natural side effect of developing this way.

Tip

Each test above corresponds to an equivalence class from the *Equivalence Partitioning* reading. TDD gives you a *process*; equivalence partitioning and boundary-value analysis help you choose *which* tests to write at each step.

Part 2

Why TDD Forces Testable Design

A Side Effect at Least as Valuable as the Tests

It is very hard to write a test-*first* test for code that is badly designed for testing.
If a piece of logic, in the middle of a function:

- reaches out to a **real database**,
- makes a **real network call**, or
- reads the **real system clock**,

writing a fast, repeatable test for it *before the code exists* forces you to confront that problem immediately — not later, when a teammate struggles to test your finished code.

Key Idea: Hard-to-Test Code Is a Design Smell

Key Idea

A function or class that is hard to unit test is usually not “fine, just annoying to test” — it is very often a symptom of a real design problem: hidden dependencies, mixed responsibilities, or global/shared state.

TDD’s real long-term value is arguably less about the tests themselves, and more about the **design pressure** that writing them first applies.

Part 3

Dependency Injection and Seams

The Problem: A Hard-Coded Dependency

A service that emails a user a password-reset code:

```
import smtplib

class PasswordResetService:
    def send_reset_email(self, user_email, reset_token):
        message = f"Subject: Password Reset\n\nYour reset code is {reset_token}"
        server = smtplib.SMTP("smtp.example.com")
        server.sendmail("noreply@example.com", user_email, message)
        server.quit()
```

This class **constructs its own connection** to a real SMTP server, deep inside the method we want to test.

Why Is This Hard to Test?

Testing `send_reset_email()` as written would require a real (or realistic fake) mail server just to run.

- **Slow** — a real network round-trip per test.
- **Flaky** — depends on an external system being up.
- **Unnecessary** — all we actually want to check is that the service builds the right message and asks the mail system to send it to the right address.

The Fix: Constructor Injection

Have the class *accept* its dependency from the outside, instead of constructing it internally:

```
class PasswordResetService:
    def __init__(self, email_client, logger):
        self.email_client = email_client
        self.logger = logger

    def send_reset_email(self, user_email, reset_token):
        message = f"Subject: Password Reset\n\nYour reset code is {reset_token}"
        self.email_client.send(user_email, message)
```

Production: a real `Smtplib.SMTP`. Tests: *any* object with a `.send(to, message)` method — no real network required.

Key Idea: Seams

Key Idea

Michael Feathers (*Working Effectively with Legacy Code*) calls a place where you can alter a program's behavior *without editing the code at that exact spot* a **seam**.

The `email_client` parameter is a seam: production code and test code can each supply a different implementation through the *exact same seam*, without `PasswordResetService` itself ever changing.

Watch For These Hard-Coded Dependencies

Tip

Common offenders worth watching for:

- direct instantiation of a **database connection**
- an **HTTP client**
- a **file handle**
- `datetime.now()` — the real system clock
- `random` — real randomness

Each one makes a unit test slower, less deterministic, or dependent on an external system.

Part 4

The Test Double Taxonomy

Five Flavors, One Running Example

Test double: any stand-in object used in place of a real dependency. Vocabulary from Gerard Meszaros, *xUnit Test Patterns* (2007): **dummy**, **stub**, **fake**, **spy**, **mock**.

Running example for the rest of this section: an `order_service` that, to place an order, normally talks to three external collaborators:

- a **payment service**
- a **database**
- an **email service**

Each is a candidate for a test double.

Dummy — “I Just Need Something to Pass In”

Required by the function/interface, but the test doesn't actually use it.

```
class DummyLogger:
    pass

def calculate_total(price, logger):
    return price * 1.06

dummy_logger = DummyLogger()
assert calculate_total(100, dummy_logger) == 106
```

The function requires `logger` but never uses it. We supply a dummy simply to satisfy the parameter.

Think: “Here is something because you require an argument.”

Dummy: Poisoning It on Purpose

Tip

A dummy can be **poisoned**, so any accidental use fails loudly instead of silently:

```
class DummyLogger:
    def log(self, message):
        raise AssertionError(
            "DummyLogger should never be called in this test")
```

The defining trait of a dummy: a *correct* test never actually calls it.

Stub — “When You Ask, I’ll Give You This Answer”

Returns a predefined response; no expectations about how it’s used.

```
class PaymentStub:
    def process_payment(self, amount):
        return True

def place_order(payment_service):
    if payment_service.process_payment(100):
        return "Order confirmed"
    return "Payment failed"

payment_stub = PaymentStub()
assert place_order(payment_stub) == "Order confirmed"
```

We never contact a real payment system. Note: the test never inspects PaymentStub afterward.
Think: “Give me a question; I’ll return the answer I’ve been programmed to give.”

Fake — “I Actually Work, But I’m Simplified”

A genuinely working, lightweight implementation — not fit for production.

```
class FakeDatabase:
    def __init__(self):
        self.users = {}

    def save_user(self, user_id, name):
        self.users[user_id] = name

    def get_user(self, user_id):
        return self.users.get(user_id)

fake_db = FakeDatabase()
fake_db.save_user(1, "Alice")
assert fake_db.get_user(1) == "Alice"
```

This is **state verification**: we check the fake’s resulting state, not the call itself. Real: PostgreSQL. Fake: a Python dict.

Think: “I’m a simplified but functional version of the real thing.”

Spy — “I’ll Remember What Happened”

Records how it was used, so the test can inspect the interaction afterward.

```
class EmailSpy:
    def __init__(self):
        self.sent_to = []

    def send_email(self, address):
        self.sent_to.append(address)

def register_user(email_service):
    email_service.send_email("alice@example.com")

email_spy = EmailSpy()
register_user(email_spy)
assert "alice@example.com" in email_spy.sent_to
```

The spy lets us ask, afterward: was an email sent? Who received it?

Think: “I’ll watch what happens and tell you afterward.”

Fake vs. Spy: A Blurry Line

Common Pitfall

FakeDatabase and EmailSpy are structurally very similar — both just record information they receive. The distinction is mostly about *intent*:

- a **fake** aims to be a believable, reusable stand-in for the real dependency;
- a **spy** exists purely to give *one* test something to inspect.

Don't lose sleep over classifying perfectly — the vocabulary is for communicating *purpose* to teammates, not rigid categorization.

Mock — “I Expect You to Interact With Me This Way”

Pre-configured with *expected* interactions; the test verifies them directly.

```
from unittest.mock import Mock

payment_mock = Mock()
payment_mock.process_payment.return_value = True

place_order(payment_mock)

payment_mock.process_payment.assert_called_once_with(100)
```

This is **behavior verification**: we’re not only interested in the result — we’re verifying that `place_order()` called `process_payment(100)` exactly once.

Think: “I expect you to call me in a particular way, and I’ll verify that you did.”

Quick Recap: Keeping the Five Straight

Tip

- **Dummy** — fills a required space.
- **Stub** — gives canned answers.
- **Fake** — actually works, but simply.
- **Spy** — records interactions.
- **Mock** — checks expectations.

Most commonly confused pair — **stub vs. mock**: a stub controls what your code *receives*; a mock verifies *how* your code interacted with a dependency.

Summary of the Taxonomy

Double	Purpose	Verifies via	Seen above as
Dummy	Fills a required parameter; never used	(nothing)	DummyLogger
Stub	Returns a canned answer	(nothing)	PaymentStub
Fake	A lightweight, working implementation	State	FakeDatabase
Spy	Records calls to inspect afterward	State	EmailSpy
Mock	Pre-configured, checked against the call	Behavior	assert_called_once_with()

Quick Check: Which Double Is This?

Quick Check — Think, Then Discuss

For each scenario, which test double fits best?

- ① A test needs an `InventoryRepository` to return a fixed stock count.
- ② A test must confirm `send_welcome_email()` was called with the new user's exact email address.
- ③ A constructor requires an `AuditLogger` that the method under test never calls.
- ④ A test suite needs a realistic, in-memory key-value store replacing a slow, real cache.

Discuss with a neighbor (~2 min), then we'll go over the answers together.

Part 5

Classical vs. Mockist Verification

Two Philosophies About What a Test Should Check

The dummy/stub/fake side and the spy/mock side reflect two schools of TDD:

- **Classical / Detroit:** Prefer real objects; use test doubles sparingly (slow, external, nondeterministic dependencies only). Checks *resulting state*.
- **Mockist / London:** Isolate the unit from *every* collaborator (typically with mocks), so a failure always points to one unit. Checks *interactions*.

The mockist approach traces to Mackinnon, Freeman & Craig's "Endo-Testing" (XP2000) and Freeman & Pryce's *Growing Object-Oriented Software, Guided by Tests* (2009). Beck's own *TDD By Example* leans classical.

Trade-offs: Neither School Is Simply “Correct”

Mockist tests

- + Isolate failures precisely
- + Run fast — nothing real instantiated
- Over-specified assertions couple tests to *how* code works, not just *what* it does

Classical tests

- + Survive refactoring better
- + More confidence components work together
- Can be slower; harder to isolate the faulty component

Where Most Real Projects Land

Tip

Most real projects land *between* the two schools:

- Use real objects and state verification for simple, fast, deterministic collaborators.
- Reach for stubs, fakes, spies, or mocks specifically at the boundaries where a dependency is **slow, external, non-deterministic, or expensive**.

Part 6

Common Pitfalls & Wrap-Up

Three Pitfalls to Avoid

Common Pitfall

Skipping the Red step. If you never watch a new test fail, you can't be sure it's capable of failing at all — a typo or tautology can pass “by accident.”

Common Pitfall

Mocking everything, out of habit. A simple, fast, deterministic value object is usually better used *for real* — mocking it adds ceremony without reducing risk.

Common Pitfall

Mocks that lie. A double is only useful if it behaves like the real dependency. spec/autospec catch some mismatches, but not every behavioral one.

A Practical Mental Model

Ask yourself:

- “What is the smallest failing test I could write first?”
- “What hard-coded dependency is hiding here, and what seam could I introduce?”
- “Do I need this object to return something (stub), behave like a lightweight real thing (fake), let me inspect it afterward (spy), or verify how it was called (mock)?”
- “Was this assertion checking *what* the code does, or *how* it does it internally?”

Key Takeaways

- ① **Red before Green.** A test you've never watched fail can't be fully trusted.
- ② **Minimum code, then refactor.** Resist building more than the current test requires.
- ③ **Hard-to-test code is usually a design smell** — hidden dependencies are the most common cause.
- ④ **Dependency injection creates seams.** Supply collaborators from outside.
- ⑤ **Five doubles, five purposes** — know which one you actually need.
- ⑥ **A double is only as useful as it is honest.**

Discussion Question

Quick Check — Think, Then Discuss

Why can heavily mocked tests be *more* likely to break during a refactor that doesn't change any externally visible behavior?

Think for a minute, then let's discuss as a class.

What We Didn't Cover Today

In the interest of realistic pacing, the following are in the reading but were **not** covered in lecture — you are responsible for them as reading:

- Full narrative history of TDD/XP and of the classical/mockist schools (reading, Sections 2 and 6 case studies).
- TDD Cycles 3–4 test code in full (reading, Section 3.2) — we summarized the pattern instead of showing every test.
- The restaurant-reservation dummy analogy (reading, Section 5.1).
- Practice Exercises 1, 2, 4, and 5, and Discussion Questions beyond today's in-class question (reading, Sections 9–10).

Questions?

Backup Slides

Reference material only — not part of the 75-minute lecture

Backup: Full History — Origins of TDD

Background

Kent Beck popularized test-driven development as part of **Extreme Programming (XP)** in *Extreme Programming Explained: Embrace Change* (1999), then devoted an entire book to it, *Test-Driven Development: By Example* (2002). Beck describes TDD not as something he invented outright, but as a rediscovery of a discipline he had seen used informally decades earlier.

Backup: Full History — Two Schools, Two Books

Background

The mockist/London approach traces to “Endo-Testing: Unit Testing with Mock Objects” by Tim Mackinnon, Steve Freeman, and Philip Craig (XP2000). Freeman later co-authored, with Nat Pryce, *Growing Object-Oriented Software, Guided by Tests* (2009) — the canonical mockist/London text, in which nearly every collaborator is mocked. Kent Beck’s own *TDD By Example* (2002) leans classical: prefer real collaborators, reach for doubles selectively.

Backup: The Restaurant Reservation Analogy

Tip

Imagine a restaurant reservation system requires a phone number, but your test only exercises the table-allocation algorithm. You might supply `phone = "000-000-0000"`: the algorithm doesn't care about the phone number, so it is a **dummy value** — present only because the function requires it.

Backup: Remaining Practice Exercises (Homework)

- **Exercise 1:** Build `calculate_tip(bill_total, party_size)` using strict Red-Green-Refactor.
- **Exercise 2:** Find the hard-coded dependency in a `SessionManager` that uses `time.time()`, and refactor with constructor injection.
- **Exercise 4:** Rewrite a mock-based (behavior) test as a state-verification test using a fake; compare trade-offs.
- **Exercise 5:** Argue against “every collaborator should always be mocked.”

Backup: Remaining Discussion Questions

- 1 Why does seeing a test fail (Red) matter, beyond confirming the feature isn't built?
- 2 In what sense is TDD a design tool, not just a testing tool?
- 3 What is a “seam,” and how does dependency injection create one?
- 4 Why might a poisoned dummy be more useful than a silent one?
- 5 Fake vs. spy, in your own words — why do practitioners disagree on the line?
- 6 Why is a mock's assertion “behavior verification” rather than “state verification”?
- 7 What risk does “mocks that lie” describe? How do `spec/autospec` help?
- 8 How would a classical developer and a mockist developer test the same class differently?
- 9 (*Covered in class*) Why can heavily mocked tests break during a behavior-preserving refactor?
- 10 How do TDD, DI, and the test-double taxonomy relate to “testing is context-dependent”?

Backup: Full Term Summary (from the Reading)

Term	Meaning
TDD	Writing a failing test before the code that makes it pass
Red-Green-Refactor	Fail, pass minimally, then clean up without changing behavior
Fake it till you make it	Passing a test with a deliberately oversimplified implementation
Dependency injection	Supplying dependencies from outside instead of constructing them internally
Seam	A place where behavior can be altered without editing that exact code
Dummy / Stub / Fake / Spy / Mock	See taxonomy summary earlier in this deck
Classical / Mockist TDD	Real objects & state checks / isolate everything & check interactions